

Algorithmic Design & Implementation Details

A linear regression model was built and optimized using full-batch gradient descent to predict concrete compressive strength given 8 features of a concrete. The loss function selected was mean-squared error. To evaluate performance, the R^2 statistic was calculated. The computed expressions are below.

$$L(\vec{y}_{obs}, \vec{y}_{pred}) = MSE(\vec{y}_{obs}, \vec{y}_{pred}) = \frac{1}{N} \sum_{i=1}^N (y_{obs,i} - y_{pred,i})^2$$

$$R^2 = 1 - \frac{MSE(\vec{y}_{obs}, \vec{y}_{pred})}{Var(y_{obs})} = 1 - \frac{\frac{1}{N} \sum_{i=1}^N (y_{obs,i} - y_{pred,i})^2}{\frac{1}{N} \sum_{i=1}^N (y_{obs,i} - \bar{y}_{obs})^2} = 1 - \frac{\sum_{i=1}^N (y_{obs,i} - y_{pred,i})^2}{\sum_{i=1}^N (y_{obs,i} - \bar{y}_{obs})^2}$$

The backpropagation implementation used the partial derivative of the loss with respect to the model weights. First, a model of $y_{pred} = \vec{m} \cdot \vec{x} + b$ was used, where $\vec{m}$, b are the model weights and intercept, and $\vec{x}$ is the input feature vector. Using this model and the definition of mean-squared error, the partial derivatives were derived.

$$\frac{\partial L}{\partial m_k} = \frac{\partial}{\partial m_k} \left[\frac{1}{N} \sum_{i=1}^N (y_{obs,i} - y_{pred,i})^2 \right] = \frac{\partial}{\partial m_k} \left[\frac{1}{N} \sum_{i=1}^N (y_{obs,i} - \vec{m} \cdot \vec{x}_i - b)^2 \right] = \frac{2}{N} \sum_{i=1}^N (y_{obs,i} - \vec{m} \cdot \vec{x}_i - b)(-x_i^{(k)})$$

$$\frac{\partial L}{\partial b} = \frac{\partial}{\partial b} \left[\frac{1}{N} \sum_{i=1}^N (y_{obs,i} - y_{pred,i})^2 \right] = \frac{\partial}{\partial b} \left[\frac{1}{N} \sum_{i=1}^N (y_{obs,i} - \vec{m} \cdot \vec{x}_i - b)^2 \right] = \frac{2}{N} \sum_{i=1}^N (y_{obs,i} - \vec{m} \cdot \vec{x}_i - b)(-1)$$

These results then could be substituted into the model weight update step.

$$m_k \leftarrow m_k - \alpha \frac{\partial L}{\partial m_k} = m_k + \frac{2\alpha}{N} \sum_{i=1}^N (y_{obs,i} - \vec{m} \cdot \vec{x}_i - b)(x_i^{(k)})$$

$$b \leftarrow b - \alpha \frac{\partial L}{\partial b} = b + \frac{2\alpha}{N} \sum_{i=1}^N (y_{obs,i} - \vec{m} \cdot \vec{x}_i - b)$$

Overall, the gradient descent took the form of the following pseudocode procedure.

Function gradient_descent($\vec{x}, \vec{y}_{obs}$)
 Set (m_k, b) randomly between (0,1)

While $R^2 \leq 0$ and $N < 10^8$
 Calculate $\vec{y}_{pred}, \frac{\partial L}{\partial m_k}, \frac{\partial L}{\partial b}$
 $m_k \leftarrow m_k - \alpha \frac{\partial L}{\partial m_k}$
 $b \leftarrow b - \alpha \frac{\partial L}{\partial b}$
 Recalculate current R^2
 $N \leftarrow N + 1$
Return (m_k, b)

The hyperparameters selected are listed in Table 1. This procedure first starts by initializing model weights to a floating-point number between 0 and 1. This choice attempts to avoid an unbiased starting point on the loss field. Parameter updates are then performed as previously described. To decide if the training has been completed, either the r-squared statistic must become positive or the number of iterations must be exceeded. Not included in pseudocode is the choice to calculate two r-squared statics: one evaluated over the training set and the other over the test set. The r-squared used for the loop conditional is the lower two statistics. This means that both the r-squared statistic for the training set and the test set must become positive before the loop terminates. This choice was made to attempt to reach a model with more predictive power. In testing, this condition was always the first to be exceeded, and the maximum iteration was never reached.

Learning rate (α)	10^{-7}
Maximum iterations (N)	10^8

Table 1: Gradient descent implementation parameters

The software environment included the following dependencies:

- Python 3.10.14
- NumPy 2.2.6
- Matplotlib 3.10.9
- Pandas 2.3.3
- Statsmodels 0.15.0
- Scikit-Learn 1.7.2

Cross-validation was used for the evaluation, both as previously described in the training phase and when creating the final model summary statistics. The exact partition of the dataset was to use 130 instances (rows 501-630) as the test set, and the remaining 900 as the training set.

For the scaling and convergence speed analysis, figures 1 and 2 were generated. Figure 1 shows the mean-squared error as a function of model training iteration, and figure 2 shows the same with normalized data. Data was normalized using the following formulation.

$$\begin{aligned} col_{min}^{(k)} &= \operatorname{argmin}_i(x_i^{(k)}) \\ col_{max}^{(k)} &= \operatorname{argmax}_i(x_i^{(k)}) \\ x^{(k)} &\leftarrow \frac{x^{(k)} - col_{min}^{(k)}}{col_{max}^{(k)} - col_{min}^{(k)}} \end{aligned}$$

Where the notation $x_i^{(k)}$ refers to the feature k of the i -th sample, and the dataset passed into the training phase has all features, including the concrete compressive strength, normalized.

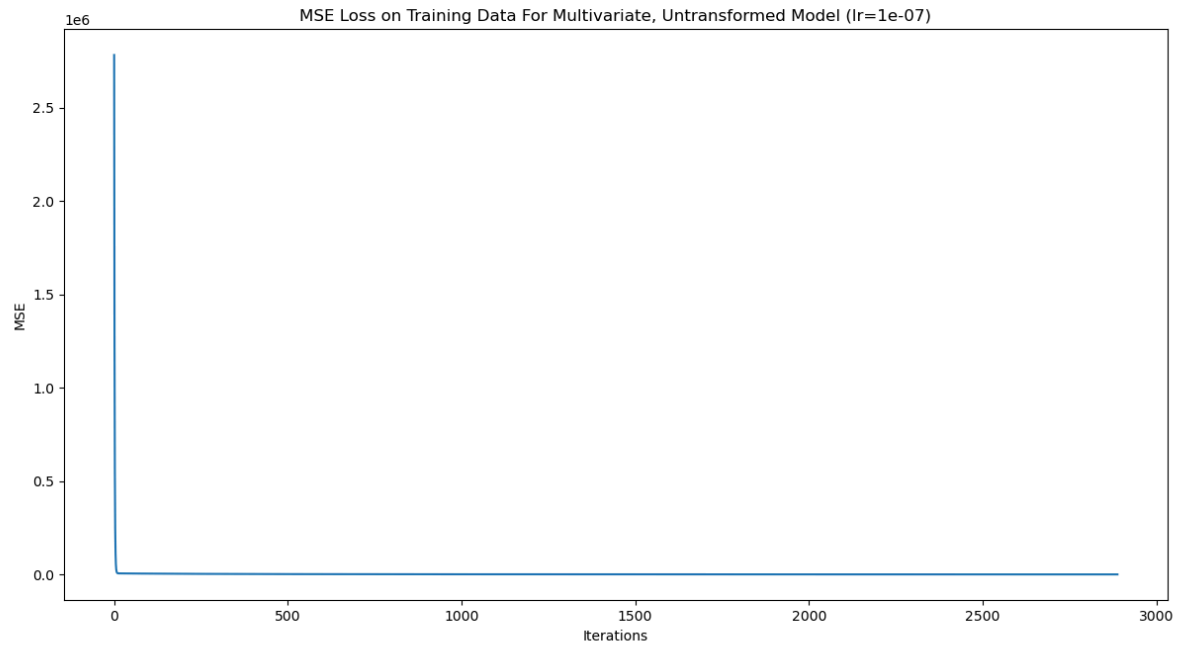


Figure 1: MSE Loss over training epoch for untransformed, linear model with learning rate 10^{-7}

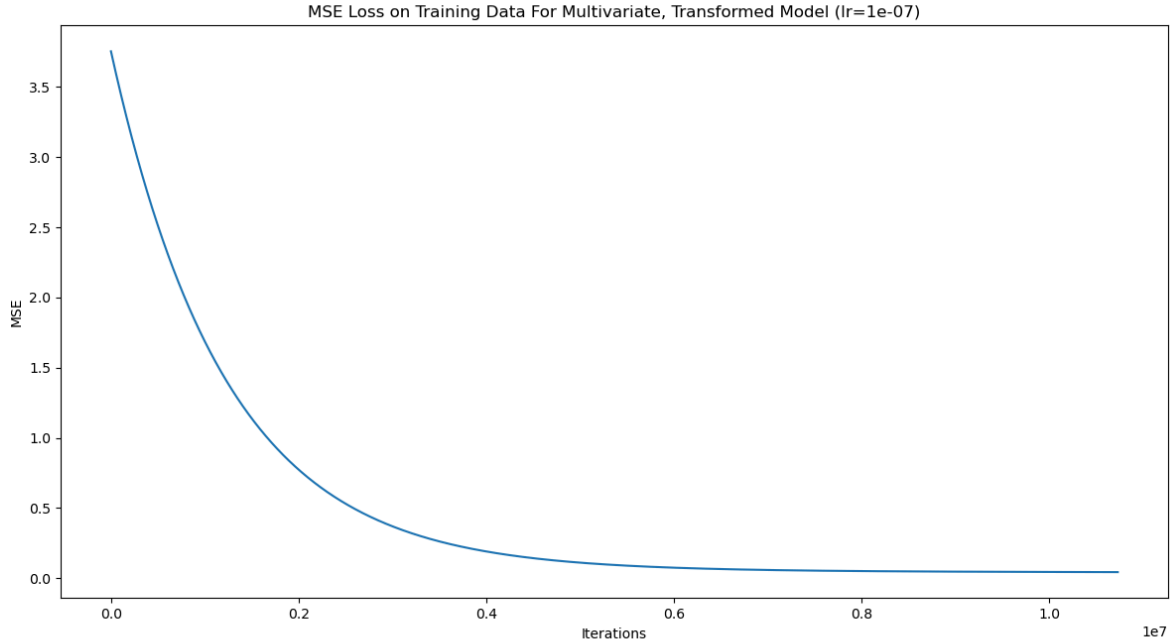


Figure 2: MSE Loss over training epoch for normalized, linear model with learning rate 10^{-7}

To generate the visualizations, the loss each iteration was stored in a list, and the visualization was plotted using Matplotlib after training was complete. The results show that significantly more compute was required to end the training phase for the normalized model than the unnormalized one, but that loss was significantly less on the normalized model. The greater number of iterations can be attributed to the significantly smaller size of the features, which directly decreases the size of each backpropagation step, even with constant learning rates. These smaller steps also allowed the gradient descent to navigate the loss landscape with more granularity and find a parameter setting that was closer to the optimal than allowed by the larger steps.

Linear vs. Non-Linear Feature Transformations

Figure 3 shows the same mean-squared error loss versus training iteration, but for a model with data nonlinearly transformed, unlike the linear normalization. Specifically, the transformation took the following form:

$$\begin{aligned}\vec{x} &\leftarrow \ln(\vec{x}) \\ y_{obs} &\leftarrow \ln(y_{obs})\end{aligned}$$

Where $\vec{x}$ is the observed input training features, and y_{obs} is the observed concrete compressive strength. After applying this transformation, the training process took $\sim 10^6$. This number of steps was less than the $\sim 10^7$ required for the normalized data, but greater than the $\sim 10^3$ required for the unnormalized data. This result checks with intuition; normalized

backpropagation limits feature values to at most 1 – limiting update step sizes proportionally – while logarithmic scaled features values do not have an upper limit on feature sizes or step sizes while still constricting their growth.

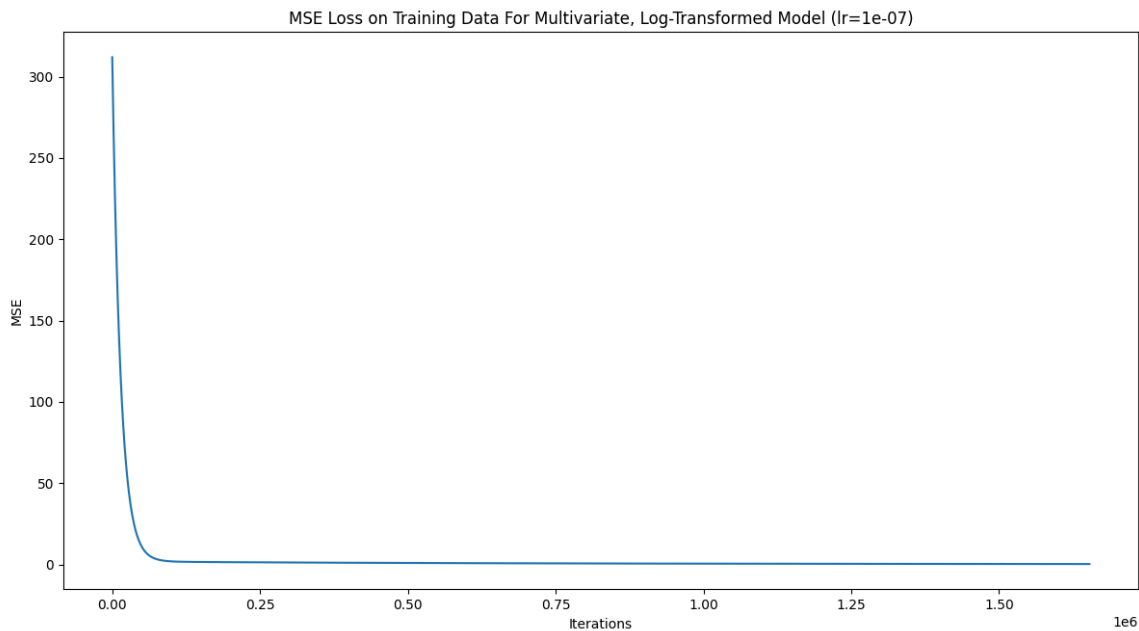


Figure 3: MSE Loss over training epoch for log-transformed, linear model with learning rate 10^{-7}

Table 2 shows the final model summary results across the untransformed, normalized, and log-transformed data transformations. The model is summarized by computing its final mean-squared error loss and R^2 across training and test sets individually. As expected, the magnitudes of the mean-squared error were greatest for the untransformed data, and smallest for the normalized data. Interestingly, the final test set mean-squared error was larger for the log-transformed data than its final training set loss. While uncommon, this is likely due to random chance from the initialization of the weights. The final R^2 statistics were greater in the training phase than the testing for the transformed and untransformed models. The untransformed model performed better, indicating that finer step sizes were not necessary to reach the local minimum it found. In correspondence to the log-transformed data's loss, the test R^2 was again worse than the training set. It also had the highest R^2 on the test set, giving it the most predictive power of the three models. The choice to stop training only once a positive R^2 was observed likely caused some sensitivity to initial conditions, as it reflects the state of the model the instant it reaches a valid statistic value, rather than its true steady state loss. Future testing may experiment with letting each model run a fixed number of steps after it achieves the threshold.

	Training MSE	Testing MSE	Training R^2	Testing R^2
Untransformed	185.0355	217.701	0.332	0.000175

Normalized	0.0405	0.0472	0.0567	3.0100e-8
Log-Transformed	0.268	0.243	1.305e-6	0.0737

Table 2: Model summary for fixed learning rate ($\alpha = 10^{-7}$) with training stopped at first instance of positive R^2 score

Univariate vs. Multivariate Model Dynamics

Table 3 shows the model coefficients for each feature across multivariate and univariate models. Coefficients were found by training a multivariate model with Scikit LinearRegression, and individual univariate models for each feature with Scikit LinearRegression. Notably, the coefficients are significantly different across models, in some cases reversing their weights sign and predicting has the opposite effect on compressive strength. The result is likely due to multicollinearity between features. One underlying assumption to linear regression is that input features are completely independent. This model breaks that assumption. For example, we'd expect water mass to go down with the addition of superplasticizer as both increase workability but only water decreases compressive strength. The combination of both variables, therefore, could represent concrete workability, which may itself be a better feature for compressive strength. For a model with perfectly independent features, the multivariate and univariate model coefficients would be identical.

Feature	Multivariate Coefficient	Univariate Coefficient
Cement	0.132845	0.0808
Blast Furnace Slag	0.12504038	0.0312
Fly Ash	0.10673935	-0.0533
Water	-0.13313546	-0.206
Superplasticizer	0.11618198	0.883
Coarse Aggregate	0.02863288	-0.0274
Fine Aggregate	0.0340842	-0.0306
Curing Age	0.11876335	0.0950

Table 3: Multivariate vs. univariate feature coefficients for Scikit LinearRegression models

Statistical Inference & Regression Analysis

Table 4 shows the p-values produced for a multivariate OLS model across different data transformations. These models were computed using the Statsmodels package. A lower p-value indicates a larger statistical significance of a feature towards impacting the compressive

strength. For example, in all cases age carried the most statistical significance, and was greatest in the log-transformed data signalling that this transformation may be a better description. It is also noted that the untransformed and normalized datasets have identical statistical significance for each feature, but both differ from the log-transformed data's p-values. This can be understood by considering the linearity of the t-test. The t-test centers each data point around its mean and normalizes by their standard deviation. Both operations are linear, so applying a linear transformation to the data appears in both the numerator and denominator, and are ultimately cancelled out, producing identical p-values. The log-transformed data is non-linear, therefore its transformations are not conserved under the t-test. As described previously, this is potentially useful in showing if a transformation is a better feature description for a model if it becomes more statistically significant after its application.

Feature	p-value	p-Value (Normalized)	p-Value (Log-Transformed)
Constant	0.380372	0.537618	0.04276
Cement	1.9e-41	1.9E-41	2.3e-101
Blast Furnace Slag	1.6e-23	1.6E-23	1.25e-34
Fly Ash	5.02e-12	5.02E-12	2.91e-06
Water	0.000201	0.000201	8.95e-15
Superplasticizer	0.00181	0.00181	1.72e-08
Coarse Aggregate	0.054425	0.054425	0.623855
Fine Aggregate	0.059491	0.059491	0.005376
Age	5.78e-82	5.78e-82	5.6e-235

Table 4: p-values for features in untransformed, linear-transformed, and log-transformed linear regression models

Code & Reproducibility Access

All code is available at the public GitHub repository
https://github.com/jonathanhouse/data_mining_project_1.

To run analysis:

1. Download the software dependencies.
 - a. Alternatively, the virtual environment in the repository can be used by running **"source sklearn-env/bin/activate"** from the command-line while inside the base directory.
2. Run **"python3 main.py --analysis [analysis_type]"** from the command-line within the base-directory

- a. Valid analysis types are “**convergence**”, “**nonlinear**”, “**univariate**”, “**statistical**”, or “**all**”. These types correspond with the analysis requirements for this project. Visualizations are shown with Matplotlib, and tables are printed to the command-line via Pandas.
- b. Note, training for “convergence” and “nonlinear” analysis performed with a maximum number of iterations of 10^8 , but due to the significant compute requirement this is modified to 10^5